

Attention 层 QKV 投影：QKVParallelLinear 原理分析

1. Attention 中 QKV 投影

Transformer Attention 计算前，需要先将输入 hidden state 投影为：

- Query (Q)
- Key (K)
- Value (V)

标准计算：

$$Q = X \times W_q$$

$$K = X \times W_k$$

$$V = X \times W_v$$

其中：

$$W_q.shape = [num_heads \times head_dim, hidden_size]$$

$$W_k.shape = [num_kv_heads \times head_dim, hidden_size]$$

$$W_v.shape = [num_kv_heads \times head_dim, hidden_size]$$

2. 为什么合并 QKV

传统实现：

分别执行 3 次矩阵乘法。

X
├── matmul(Wq)
├── matmul(Wk)
└── matmul(Wv)

需要：

3 次 GEMM

为了减少：

- kernel launch
- memory access
- GPU 调度开销

通常将 QKV 合并为一个大矩阵：

$\text{merged_weight} = [W_q; W_k; W_v]$

只执行一次：

$\text{output} = X \times \text{merged_weight}^T$

然后切分：

$Q = \text{output}[:Q_size]$

$K = \text{output}[Q_size:Q_size+K_size]$

$V = \text{output}[Q_size+K_size:]$

3. 初始化过程

代码：

$\text{self.num_heads} = \text{total_num_heads} / \text{tp_size}$

$\text{self.num_kv_heads} = \text{total_num_kv_heads} / \text{tp_size}$

其中：

$\text{tp_size} = \text{tensor parallel 数量}$

例如：

$\text{hidden_size} = 4096$

$\text{total_num_heads} = 32$

$\text{total_num_kv_heads} = 8$

$\text{head_size} = 128$

$\text{tp_size} = 2$

则：

$\text{num_heads} = 16$

$\text{num_kv_heads} = 4$

4. 输出维度计算

代码：

$\text{output_size} = (\text{total_num_heads} + 2 * \text{total_num_kv_heads}) * \text{head_size}$

原因：

Q 需要：

$$32 \times 128 = 4096$$

K 需要:

$$8 \times 128 = 1024$$

V 需要:

$$8 \times 128 = 1024$$

总大小:

$$4096 + 1024 + 1024 = 6144$$

即:

`merged_weight.shape = [6144, 4096]`

5. 合并后的内存布局

按 output dimension 拼接:

`merged_weight`

Q weight
[4096,4096]
K weight
[1024,4096]
V weight
[1024,4096]

即:

```
merged_weight = concat(  
    Wq,  
    Wk,  
    Wv,  
    dim=0  
)
```

6. Tensor Parallel 切分

假设:

tp_size = 2

每张 GPU 只保存一部分：

GPU0:

Q = [2048,4096]

K = [512,4096]

V = [512,4096]

GPU1:

Q = [2048,4096]

K = [512,4096]

V = [512,4096]

每卡总大小：

3072×4096

7. weight_loader 加载逻辑

checkpoint 中：

q_proj.weight

k_proj.weight

v_proj.weight

分别调用：

weight_loader(...,"q")

weight_loader(...,"k")

weight_loader(...,"v")

8. Q 权重加载

代码：

if loaded_shard_id == "q":

计算：

shard_size = num_heads $\times$ head_size

例如：

$$16 \times 128 = 2048$$

offset:

shard_offset = 0

表示:

Q 在 merged_weight 开头。

9. K 权重加载

代码:

elif loaded_shard_id == "k":

计算:

shard_size = num_kv_heads $\times$ head_size

例如:

$$4 \times 128 = 512$$

offset:

shard_offset = num_heads $\times$ head_size

即:

2048

表示:

K 紧跟在 Q 后面。

10. V 权重加载

代码:

else:

计算:

shard_size = num_kv_heads $\times$ head_size

即：

512

offset：

shard_offset =

num_heads × head_size

• num_kv_heads × head_size

=

2048 + 512

=

2560

表示：

V 位于 K 后面。

11. narrow 定位写入区域

代码：

```
param_data = param_data.narrow( dim=0, start=shard_offset, length=shard_size )
```

GPU 本地参数布局：

Q shard [2048,4096] offset=0
K shard [512,4096] offset=2048
V shard [512,4096] offset=2560

12. checkpoint 权重切分

代码：

```
loaded_weight = loaded_weight.chunk( tp_size, dim=0 )[tp_rank]
```

例如：

Q weight:

[4096,4096]

切分:

GPU0 → [2048,4096]

GPU1 → [2048,4096]

K weight:

[1024,4096]

切分:

GPU0 → [512,4096]

GPU1 → [512,4096]

V weight 同理。

13. copy 到本地参数

代码:

```
param_data.copy_(loaded_weight)
```

完成:

checkpoint → GPU local merged weight

数据写入。

14. 最终 GPU 本地布局

GPU0:

Q local shard
[2048,4096]
K local shard
[512,4096]
V local shard
[512,4096]

总大小：

[3072,4096]

15. 推理阶段一次 GEMM

原来：

$$Q = X \times W_q$$

$$K = X \times W_k$$

$$V = X \times W_v$$

需要：

3 次 GEMM

优化后：

$$\text{output} = X \times \text{merged_weight}^T$$

一次 GEMM

然后 split：

$$Q = \text{output}[:, 2048]$$

$$K = \text{output}[2048:2560]$$

$$V = \text{output}[2560:]$$

总结：

QKVParallelLinear 本质：

将 Attention 中 Q、K、V 三个线性层按输出维度合并，结合 Tensor Parallel 对输出维度切分，每个 GPU 保存局部权重，推理时通过一次 GEMM 完成 QKV 投影计算。